

Exploring Columnar Execution in DuckDB through a Sales Analytics Application

Yunhui Dong

1. Chosen Database System and Motivation

For this project, I have chosen the database to be DuckDB. DuckDB is an analytical database, and it is particularly designed for OLAP operations. It is a column-based database with support for vectorized query execution. Unlike row-based databases like MySQL, DuckDB is particularly designed for analytical queries with aggregation and scanning operations. I have chosen DuckDB for this particular project because it is a modern analytical database. Its internal design is well documented, and it is particularly suited for data analysis and data science. Since this is a one person team, I will be able to focus on one core internal mechanism.

2. Planned Internal Focus Areas

The main focus of this project is column storage and vectorized query execution. I'll go into more detail about how DuckDB stores its data by columns instead of rows, and how this allows it to only read the columns it needs during a query. I'll also go into more detail about how DuckDB executes queries by batches of vectors instead of one row at a time, specifically by examining how it executes aggregation queries internally. This type of project is suitable for a one person team because it focuses on one major aspect of database architecture instead of trying to understand many different parts of a database.

3. Preliminary Application Idea

For the application part, I would like to develop a simple sales analytics application using Python and DuckDB. This application will import a CSV file into the DuckDB, then execute several analytical queries on it, such as GROUP BY, filtered aggregations, and range queries. The application will have a very basic interface, possibly a command-line one, as the focus here is not to create a great user experience but to demonstrate how the database's internal workings affect the application's behavior. The data can be real or synthetic and will have to be sufficient to simulate a proper analytical workload.

A key part of this project is showing how the app's actions map to how the database works inside. So if the app does something like "run a query like total sales by region," I will show how DuckDB only reads the necessary columns, works with small chunks of data, and does the grouping with its vectorized methods. If the query had a filter, I will show how this reduces memory usage and makes it go faster with predicate pushdown and pruning columns. I will state for each main action what the app does, what DuckDB does inside, and how this makes it go faster.

4. Team Information and Responsibilities

The tasks that I will handle include installing and configuring DuckDB, learning how it works, building the application, running experiments, and writing a final report. The project will be based on a single core internal area, as is necessary for a one group team.

5. Initial References

Raasveldt, Mark, and Hannes Mühleisen. "DuckDB." *Proceedings of the 2019 International Conference on Management of Data*, 25 June 2019,

<https://doi.org/10.1145/3299869.3320212>.

"Why DuckDB." *DuckDB*, duckdb.org/why_duckdb.